Assignment 1 - Rotation Control

In this assignment you will explore different methods for controlling rotation, and measure your efficiency at specifying parameters to match a random set of orientations. Through completing and running this assignment, you will build intuition about rotations and their specification.

Getting Started

It is recommended that you install Anaconda3, and in some cases you may need to start the anaconda prompt with administrator privileges to be able to install modules.

You should use a *clean* environment to help you avoid problems! Create and activate an environment specific to this course with the following commands, and note that you will be able to tell vscode (or your IDE of preference) to use this environment. Use python 3.11 for best compatibility.

```
conda create --name comp557f25 python=3.11
conda activate comp557f25
```

There are several requirements for the assignment, including `moderngl` for opengl bindings, `pyqt5` for windowing and UI, `pyglm` for graphics math helpers, `trimesh` for loading geometry, and `matplotlib` for plotting histograms. Use the following command to install the requirements:

```
python -m pip install -r requirements.txt
```

See the hello_* examples provided in the course content and discussed in class for a preliminary information on the moderngl opengl bindings and qt5 window and UI building library. See also the following user guides for documentation:

- pygml
- pyglm functions
- moderngl
- pyqt5

Of the links above, you will likely find the pyglm links the most important.

Provided Code

The provided code includes a main application `a1_app.py` , some helper classes `helpers.py` for a slider widget and a timer, and a set of rotation controllers that you must implement in `controllers.py` . There is also a histogram plotting application `a1_plot_histogram.py` that you can run once you've completed the assignment.

Objectives

The main application sets up two drawing canvases: a main canvas, a target rotation canvas. The main canvas and target canvas both show a coordinate frame, with the axes colour coded to show the X Y and Z axis in red green and blue respectively. You will use rotation controls in the UI and mouse movements in the main canvas (for 2 of the controller types) to try to match the orientation of the frame that you see in the target canvas. Once you've matched the orientation within 10 degrees, the screen background will turn green, and the time will be recorded in a log file called `timing.txt` .

Note that each rotation controller extends a base Controller class. It has a name, and should return a `glm.mat4` that will be updated via the UI and mouse interactions in the case of the XYBall and TrackBall.

This assignment has several steps where the marks associated with different steps are below. See likewise comments labeled `TODO` in the `controllers.py` file.

1. **(1 mark) XYZ rotation.**
   - Implement the `XYZController` . Use the parameters in the provided code to compute the appropriate rotation. Look at the glm reference to see what functions you might use.
   - Note that with XYZ order, points should be first transformed by the Z rotation, then the Y rotation, and finally the X rotation, i.e., points should be transformed by the matrix XYZ, where X, Y, and Z are matrices.
2. **(1 mark) XYX rotation.**
   - Implement the `XYXController` in a manner similar to the XYZ controller.
3. **(1 mark) Collect Samples for XYZ and XYX (1 mark).**
   - Having created controls for XYZ and XYX rotation, use the controls to match random target rotations. When the rotation matches within 10 degrees, the timer will pause, the background will become green, and the time recorded to the log file. At this point you can press "Restart Timer" button to move onto the next target (you can also use this button to skip to a new target, or start fresh). Collect at least 10 samples for each. You can note the total sample count at the top of the control pane.
4. **(1 mark) WorldLeftController.**
   - Complete the `WorldLeftController` by computing the current total rotation as a product of canonical rotation from a slider being modified and the accumulated rotation so far. As implied by the name, the slider controlled rotation should be left multiplied and can be seen as rotation aligned with the world axis.
   - When the `SliderControl` calls a given callback to provide the new value, check the *other* sliders to see if they were modified (i.e., nonzero). If you find a nonzero value, accumulate the rotation of that slider into the `accumulated_rotation` and set the slider to zero.
   - You should see that with left multiplies, your rotations are always about the screen aligned coordinate frame (x is horizontal to the right, y is up, and z is coming out of the screen).
   - Collect at least 10 samples.
5. **(1 mark) LocalRightController**
   - Complete the LocalRightController`, similar to the World/Left controller except that the rotation associated with the currently modified slider will right multiply the accumulated rotation to give the current total rotation.

- Again, be sure to accumulate other rotations and zero the other sliders if they are not zero.
- With right multiplies, you should see that changing the X rotation amount will cause the axis to spin around its X axis, regardless how it is oriented on the screen (the same is true for the other axes).
- Collect at least 10 samples

6. **(2 mark) QuatController.**

- Implement the `QuatController` by creating 4 sliders that run from -1 to 1 with a scale of 0.01. Each should call an update method which will set the given component to the desired value while adjusting the scale of the other components such that the quaternion remains unit length (and the sliders should have their values updated accordingly with `setValue` calls.
- For instance, if the real part of the quaternion is to have a given `value` as set by the slider, then the norm of the other parts should be the square root of one minus the value squared.
- Should the norm of the other components be small (i.e., zero, or close to zero), set them all to be the same value while meeting the constraint that the quaternion must be unit length.
- Collect at least 10 samples.

7. **(1 mark) XYBallController.**

- Write code to accumulate rotations about the screen X and Y axis based on dragging the mouse in the canvas. See that `mousePressEvent` and `mouseMoveEvent` stubs are provided for you.
- When you drag the mouse to the right the screen mouse x position will increase, and you should produce a rotation about the canvas' y axis such that closer parts of the 3D model move from left to right (while anything farther than the center of rotation will move from right to left on the screen).
- When you drag the mouse from the top to the bottom, the mouse y position will increase, and you should produce a rotation about the canvas' x axis. Part of the model that are in front of the center of rotation will move downward on the screen, while those parts behind the center of rotation will move up.
- While rotations do not commute, the rotations produced by your mouse movements will be small enough so that if you have a mouse motion in both X and Y then you can apply one rotation followed by the other (make a choice, or try both, but you will likely not be able to tell the difference!).
- Collect at least 10 samples.

8. **(2 marks) TrackBallController.**

- Implement the TrackBall (ArcBall) as discussed in class. Implement the helper method `get_sphere_coordinates` to map mouse positions to unit length vectors on the ball, and see that the stub shows you how to get the width and height of of the opengl wiget from the context viewport.
- Handle failure cases such as when the axis you compute has effectively zero length. Such a case corresponds to zero rotation. Other error cases can arise when computing the angle because of floating point precision.
- You can find more information in your class notes and in the paper "ARCBALL: A User Interface for Specifying Three-Dimensional Orientation Using a Mouse", a paper inGraphics Interface 1992 by Ken Shoemake, available in the course content resources.
- Collect at least 10 samples.

Finished?

Great! Submit your python implementation and log file, and optionally a readme file as a `zip` archive to MyCourses (only use `zip` format). Use the comments at the top of the files you submit to list your name and student number. Recall the readme can be used to request a late penalty waiver following the mechanism described in the first lecture slides.

Note that you are encouraged to discuss assignments with your classmates, but not to the point of sharing code and answers. All code and written answers must be your own. Please see the course outline and the fine print in the slides of the first lecture.